Space lower bounds for pseudo-deterministic missing-item algorithms even with a random oracle

Amit Chakrabarti, Manuel Stoeckl

doi:10.4230/LIPIcs.CCC.2024.28; CC BY-SA4.0

Editorial note. Original source: Amit Chakrabarti and Manuel Stoeckl, arXiv:2310.03634v2 (2024-07-03), https://arxiv.org/abs/2310.03634v2. License: Creative Commons Attribution-ShareAlike4.0, https://creativecommons.org/licenses/by-sa/4.0/.

Editorial note. Changes: selected complete author theorem/proof/context excerpts and original diagram/macros/bibliography in a standalone layout; omitted unrelated sections and author frontmatter. No mathematical corrections or additional proofs. This adapted presentation retains CC BY-SA4.0.

Allowed standard prerequisites (editorial).

- One-way avoid communication lower bound. Author Theorem lem:avoid-lb explicitly cites ChakrabartiGS22; exact problem and statement retained.

Source reference. Original section 2.4: number-theory/sources/ccc-2024-28/native/main-content.tex, source lines 758–758. Optional introductory overview; all canonical-output algorithm and proof steps are included.

Source reference. Original section 2.2: number-theory/sources/ccc-2024-28/native/main-content.tex, source lines 615–615. Optional reference used only when introducing the avoid problem; the exact problem and cited bound are retained.

In this paper, we prove that for certain parameter regimes, MissingItemFinding (henceforth, MIF) is such a problem. In the problem MIF(n, ℓ), the input is a stream $\langle e_1, \ldots, e_\ell \rangle$ of ℓ integers, not necessarily distinct, with each $e_i \in \{1, \ldots, n\}$, where $1 \leq \ell \leq n$. The goal is as follows: having received the ith integer, output a number v in $\{1, \ldots, n\} \setminus \{e_1, \ldots, e_i\}$. We will be mostly interested in the setting $\ell = o(n)$, so the "trivial" upper bound on the space complexity of MIF(n, ℓ) is $O(\ell \log n)$, achieved by the deterministic algorithm that simply stores the input stream as is.

0.1 Groundwork for Our Results

To state our results about MIF, we need to introduce some key terminology. Notice that MIF is a *tracking problem*: an output is required after reading each

input.[1] Thus, we view streaming algorithms as generalizations of finite state (Moore-type) machines. An algorithm $\mathcal{A}$ has a finite set of states Σ (leading to a space cost of $\log_2 |\Sigma|$), a finite input set $\mathcal{I}$, and a finite output set $\mathcal{O}$. It has a transition function $T\colon \Sigma \times \mathcal{I} \times \mathcal{R} \to \Sigma$ indicating the state to switch to after receiving an input, plus an output function $\gamma\colon \Sigma \times \mathcal{R} \to \mathcal{O}$ indicating the output produced upon reaching a state. How the final parameter (in $\mathcal{R}$) of T and γ is used depends on the type of randomness. We consider four cases, leading to four different models of streaming computation.

- *Deterministic.* The initial state of the algorithm is a fixed element of Σ, and T and γ are deterministic (they do not depend on the parameter in $\mathcal{R}$).
- *Random seed.* The initial state is drawn from a distribution $\mathcal{D}$ over Σ, and T and γ are deterministic. This models the situation that all random bits used count towards the algorithm's space cost.
- *Random tape.* The initial state is drawn from a distribution $\mathcal{D}$ over Σ.[2] The space $\mathcal{R}$ is a sample space; when the algorithm receives an input $e \in \mathcal{I}$ and is at state $\sigma \in \Sigma$, it chooses a random $\rho \in \mathcal{R}$ independent of all previous choices and moves to state $T(e, \sigma, \rho)$. However, γ is deterministic.[3] This models the situation that the algorithm can make random decisions at any time, but it cannot remember past random decisions without recording them (which would add to its space cost).
- *Random oracle.* The initial state is fixed; $\mathcal{R}$ is a sample space. A specific $R \in \mathcal{R}$ is drawn at the start of the algorithm and stays the same over its lifetime. When the algorithm is at state σ and receives input e, its next state is $T(e, \sigma, R)$. The output given at state σ is $\gamma(\sigma, R)$. This models the situation that random bits are essentially "free" to the algorithm; it can read from a long random string which doesn't count toward its space cost and which remains consistent over its lifetime. A random oracle algorithm can be interpreted as choosing a random deterministic algorithm, indexed by R, from some family.

These models form a rough hierarchy; they have been presented in (almost) increasing order of power. Every z-bit (2^z-state) deterministic algorithm can be implemented in any of the random models using z bits of space; the same holds for any z-bit random seed algorithm. Every z-bit random tape algorithm has a corresponding $(z + \log \ell)$-bit random oracle algorithm—the added space cost is because for a random oracle algorithm to emulate a random tape algorithm, it must have a way to get "fresh" randomness on each turn.[4]

[1] We do not consider algorithms with a "one-shot" guarantee, to only be correct at the end of the stream, because a) the adversarial setting requires tracking output b) for MIF and most other problems the difference in space complexity is generally small.

[2] Requiring that this model use a fixed initial state could make some algorithms use one additional "INIT" state.

[3] Alternatively, we could associate a *distribution* of outputs to each state, or a function mapping (input, state) pairs to outputs. As these formulations are slightly more complicated to prove things with, and only affect the space usage of MISSINGITEMFINDING algorithms by an additive $O(\log n + \log \frac{1}{\delta})$ amount, we stick with the one state = one output convention.

[4] An alternative, which lets one express z-bit random tape algorithms using a z-bit random oracle variant, is to assume the random oracle algorithm has access to a clock or knows the position in the stream for free; both are reasonable assumptions in practice.

Streaming algorithms are also classified by the kind of correctness guarantee they provide. Recall that we focus on "tracking" algorithms [BJWY20]; they present an output after reading each input item and this *entire sequence* of outputs must be correct. Here are three possible meanings of the statement "algorithm $\mathcal{A}$ is δ-error" (we assume that $\mathcal{A}$ handles streams of length ℓ with elements in $\mathcal{I}$ and has outputs in $\mathcal{O}$):

- *Static setting.* For all inputs $\tau \in \mathcal{I}^\ell$, running $\mathcal{A}$ on τ produces incorrect output with probability $\leq \delta$.
- *Adversarial setting.* For all (computationally unbounded) adaptive adversaries α (i.e., for all functions $\alpha\colon \mathcal{O}^\star \to \mathcal{I}$),[5] running $\mathcal{A}$ against α will produce incorrect output with probability $\leq \delta$.
- *Pseudo-deterministic setting.* There exists a canonical output function $f\colon \mathcal{I}^\star \to \mathcal{O}$ producing all correct outputs so that, for each $\tau \in \mathcal{I}^\ell$, $\mathcal{A}(\tau)$ fails to output $f(\tau)$ with probability $\leq \delta$.

Algorithms for the static setting are called "classic" streaming algorithms; ones for the adversarial setting are called "adversarially robust" streaming algorithms. All pseudo-deterministic algorithms are adversarially robust, and all adversarially robust algorithms are also classic.

As a consequence of Newman's theorem [New91], any random oracle or random tape algorithm in the static setting with error δ can be emulated using a random seed algorithm with only ε increase in error and an additional $O(\log \ell + \log\log |\mathcal{I}| + \log \frac{1}{\varepsilon\delta})$ bits of space. However, the resulting algorithm is non-constructive.

1 Preliminaries

Notation. Throughout this paper, $\log x = \log_2 x$, while $\ln x = \log_e x$. The set $\mathbb{N}$ consists of all positive integers; $[k] := \{1, 2, \ldots, k\}$; and $[a, b)$ is a half open interval of real numbers. For a condition or event E, the symbol $\mathbb{1}_E$ takes the value 1 if E occurs and 0 otherwise. The sequence (stream) obtained by concatenating sequences a and b, in that order, is denoted $a \circ b$. For a set S of elements in a totally ordered universe, $\textsc{sort}(S)$ denotes the sequence of elements of S in increasing order; $\binom{S}{k}$ is the set of k-element subsets of S; and $\textsc{seqs}(S, k) = \{\textsc{sort}(Y) : Y \in \binom{S}{k}\}$. We sometimes extend set-theoretic notation to vectors and sequences; e.g., for $y \in [n]^t$, write $y \subseteq S$ to mean that $\forall i \in [t] : y_i \in S$. For a set X, $\triangle[X]$ denotes the set of probability distributions over X, while $A \in_R X$ indicates that A is chosen uniformly at random from X. When naming probability distributions, we will use either calligraphic letters (e.g., $\mathcal{A}, \mathcal{D}$) or the letter μ; $A \sim \mu$ means that A is drawn from the distribution μ.

As outlined in Original section 2.2, $\textsc{avoid}(m, a, b)$ is a one-way communication problem, wherein player Alice has a set $A \in \binom{[m]}{a}$, and should send a short message to player Bob, who should use the message to output a set $B \in \binom{[m]}{b}$ that is disjoint from A. In the randomized δ-error setting, this disjointness should hold with probability $\geq 1 - \delta$.

[5] By the minimax theorem, it suffices to consider deterministic adversaries.

Theorem 1.1 (AVOID communication lower bound, from [CGS22])**.** *Suppose there exists a randomized protocol for* AVOID(m, a, b)*, in which Alice communicates $\leq K$ bits, that is δ-error either on a worst-case input or when Alice's input is chosen uniformly at random from $\binom{[t]}{a}$. Then*

$$K \geq \frac{ab}{t \ln 2} + \log(1 - \delta)\,.$$

Lemma 1.2 (Error amplification by majority vote)**.** *Let $\varepsilon \leq \delta \leq 1/3$. Say X is a random variable, and v a value with $\Pr[X = v] \geq 1 - \delta$. If $X_1, \ldots, X_p$ are independent copies of X, then the most common value in $(X_1, \ldots, X_p)$ will be v with probability $\geq 1 - \varepsilon$, for $\varepsilon = (2\delta)^{p/30}$.*

The above is a standard lemma, useful for trading error for space for algorithms with a single valid output.

Proof of Theorem 1.2. For each $i \in [p]$, let Y_i be the random indicator variable for the event that $X_i \neq v$. Let $\alpha = \frac{1}{2\delta} - 1$.The probability that v is not the most common element can be bounded by the probability that it is the not the majority element; by a Chernoff bound, this is:

$$\begin{aligned}
\Pr[\sum_{i\in[p]} Y_i \geq \frac{1}{2}p] = \Pr[\sum_{i\in[p]} Y_i \geq (1+\alpha)\delta p] &\leq \exp\left(-((1+\alpha)\ln(1+\alpha) - \alpha)\delta p\right) \\
&\leq \exp\left(-0.073((1+\alpha)\ln(1+\alpha))\delta p\right) \qquad \text{since } \alpha \geq 1/6 \\
&\leq \exp\left(-\frac{0.073}{2\delta}\ln\frac{1}{2\delta}\delta p\right) = (2\delta)^{0.036p}\,.
\end{aligned}$$

□

2 The Pseudo-Deterministic and Random Seed Lower Bounds

In this section, we prove a space lower bound for pseudo-deterministic streaming algorithms; in particular, for the most general (random oracle) type of them. See Original section 2.4 for a high-level plan of the proof.

Let $\mathcal{A}$ be a random-oracle pseudo-deterministic algorithm for MIF(n, ℓ) using z bits of state, which has worst case failure probability $\delta \leq \frac{1}{3}$. Let $\Pi\colon [n]^\star \to [n]$ be the function giving the *canonical output* of $\mathcal{A}$ after processing a stream (as was defined in Section 0.1), and let $S = \Pi([n]^\ell)$ be the set of all canonical outputs at time ℓ (for this proof we can ignore outputs at times $< \ell$). Clearly $|S| \leq n$, and we will also prove (see Theorem 2.8) that $|S| \leq 2^{z+1}$. It is easy to see that $|S| \geq \ell + 1$ and that, since MIF(n, ℓ) is nontrivial, correct algorithms must have $z \geq 1$.

The main proof in this section only applies to algorithms with very low error (potentially as small as $1/n^{\Omega(\log n)}$). To ensure that we are working with an algorithm with error this small, we will first apply Theorem 1.2, using p independent instances of $\mathcal{A}$, where $p \geq 1$ is an integer chosen later. This will produce an ε-error algorithm $\mathcal{B}$ that uses zp bits of space, where $\varepsilon \leq (2\delta)^{p/30}$. Moreover, the canonical outputs of $\mathcal{B}$ will still be given by Π.

We may view $\mathcal{B}$ as a distribution over deterministic streaming algorithms; each such algorithm, obtained by fixing $\mathcal{B}$'s oracle random string, produces

outputs according to a function of the form $A\colon [n]^\star \to [n]$. Since $\mathcal{B}$ is pseudo-deterministic, we have the following property:

$$\forall x \in [n]^{\leq \ell} : \Pr_{A \sim \mathcal{B}}[A(x) \neq \Pi(x)] \leq \varepsilon\,. \tag{1}$$

It is time to give the details of the key procedure FindCommonOutputs (abbreviated as fco), outlined earlier in Original section 2.4, that underpins our proof. The input stream positions, from 1 to ℓ, are split into $d = \Theta(\ell/(zp))$ consecutive intervals, of lengths $t_d, t_{d-1}, \ldots, t_1$, in order, where $t_d = t_{d-1} = \cdots = t_2 = \Theta(zp)$ and $t_1 = \ell - \sum_{k=2}^{d} t_k$; the constants are chosen such that $t_1 \geq \ell/2$. A call to fco takes the form $\text{fco}(B, C, x, k)$, where $B\colon [n]^\ell \to [n]$ is an output function, C is a random collection (matrix) of thresholds, $k \in [d]$, and $x \in [n]^{t_d + \cdots + t_{k+1}}$ is a stream prefix. (Notice that when $k = d$, we must have $x = \epsilon$.) By design fco is recursive, bottoming out at $k = 1$, and always returns a subset of S of cardinality w_k, where

$$w_k := 2^{k-1}(t_1 + 1)\,. \tag{2}$$

The precise logic of fco is given in Listing 1. Throughout this section, we use $\text{seqs}(Z, t)$ to denote the set of all length-t *sorted sequences* of *distinct* elements of Z. Note that

$$|\text{seqs}(Z, t)| = \binom{|Z|}{t}\,.$$

A key property of fco is that for all $x \in [n]^{t_d + \cdots + t_{k+1}}$, if C is chosen uniformly at random from $[1, 2)^{d \times \mathbb{N}}$, and $A \sim \mathcal{B}$, then w.h.p. $\text{fco}(A, C, x, k)$ produces the same set as $\text{fco}(\Pi, C, x, k)$. Another important property is that $\text{fco}(\Pi, C, x, k)$ always produces w_k distinct elements in S without a failure (i.e., returns using Algorithm 1, not Algorithm 1). These properties are formally proved in Theorem 2.2, to follow. In particular, $\text{fco}(\Pi, C, \epsilon, d)$ produces w_d distinct elements in S, showing that $|S| \geq w_d$. Since w_d is a function of z, combining this with upper bounds on $|S|$ lets us solve for a lower bound on z.

We elaborate on how we achieve the two key properties of fco. First, to ensure $\text{fco}(\Pi, C, x, k)$ equals $\text{fco}(A, C, x, k)$ for random $A \sim \mathcal{B}$ and $C \in_R [1, 2)^{d \times \mathbb{N}}$, we use a standard random threshold trick when computing the set P_h on Algorithm 1. The recursive calls to $\text{fco}(A, C, x \circ y, k - 1)$ on Algorithm 1 do not always match the outputs of $\text{fco}(\Pi, C, x \circ y, k - 1)$; as a result, the element frequency vector $f^{(h)}$ from Algorithm 1 may have random noise when computed using A instead of using Π. If Algorithm 1 used a fixed threshold value, then there would exist pseudo-deterministic mif algorithms yielding element frequencies close to this threshold, where even low-magnitude noise could affect which elements are included in P_h. Using C to choose random threshold values that are independent of the choice of A ensures that most of the time, the noise has no influence on P_h; ultimately, ensuring that $\text{fco}(\Pi, C, x, k)$ and $\text{fco}(A, C, x, k)$ most likely produce the same output. More detail is given in the proof of Theorem 2.6.

Second, the design of Listing 1 ensures that $\text{fco}(\Pi, C, \epsilon, d)$ actually produces a set of w_d possible outputs. Recall the $\text{avoid}(m, a, b)$ communication problem described in Original section 2.2. The output sets T_y of size w_{d-1} computed on Algorithm 1, for each $y \in \text{seqs}(S, t_d)$, will have a similar structure to the input and output sets for an avoid protocol, in that the set T_y is typically disjoint from y, and in that the distribution of possible T_y values is limited

Listing 1 The procedure to compute a set for Theorem 2.2

Let $t_1, \dots, t_d$, $w_1, \dots, w_d$ be integer parameters, and S the set of valid outputs

FindCommonOutputs(B, C, x, k) ▷ *abbreviated as* FCO(B, C, x, k)

▷ *Inputs: function* $B\colon [n]^\ell \to [n]$*, matrix* $C \in [1,2)^{d \times \mathbb{N}}$*, stream prefix* $x \in [n]^{t_k + \dots + t_1}$ ◁

▷ *Output: a subset of* S *of size* w_k ◁

if $k = 1$ **then**
  $e_0 \leftarrow B(x \circ \langle 1, 1, \dots, 1 \rangle)$
  for i in $1, \dots, t_1$ **do**
    $e_i \leftarrow B(x \circ \langle e_0, \dots, e_{i-1}, 1, \dots, 1 \rangle)$
  if $e_0, \dots, e_{t_1}$ are all distinct **then**
    return $\{e_0, e_1, \dots, e_{t_1}\}$ ▷ *identify* w_1 *distinct possible outputs*
  return arbitrary subset of S of size w_1 (failure)
else
  for each $y \in \text{SEQS}([n], t_k)$ **do**
    $T_y \leftarrow$ FindCommonOutputs$(B, C, x \circ y, k-1)$ ▷ *note* $|T_y| = w_{k-1}$
  $Q_0 \leftarrow T_{\langle 1,2,\dots,t_k \rangle}$
  for h in $1, 2, 3, 4$ **do**
    ▷ *gather statistics and find common elements among the sets* T_y ◁
    for each $j \in S$ **do**
      $f_j^{(h)} \leftarrow |\{y \in \text{SEQS}(Q_{h-1}, t_k) : j \in T_y\}|$ ▷ *count frequencies*
    $\theta \leftarrow C_{k,h} w_{k-1} / (16|S|)$ ▷ *set random threshold*
    $P_h \leftarrow \left\{ j \in S : f_j^{(h)} \geq \theta \binom{|Q_{h-1}|}{t_k} \right\}$ ▷ *identify "sufficiently common" elements*
    $Q_h \leftarrow Q_{h-1} \cup P_h$
    if $|Q_h| \geq w_k$ **then**
      return the w_k smallest elements in Q_h
  return arbitrary subset of S of size w_k (failure)

(through the requirement that Π agrees with a mixture of functions corresponding to deterministic streaming algorithms). Note that we cannot just extract the "most common values" that occur in the sets T_y for $y \in \text{SEQS}(S, t_d)$, because if $t_d w_{d-1} \ll |S|$, it is possible that these are very concentrated; for example there could exist a set V of size w_{d-1} so that if $y \cap V = \varnothing$, then $T_y = V$, in which case an algorithm satisfying the first property can only reliably identify w_{d-1} outputs. Instead, we iteratively build up a set of common outputs, using the following observation. If the current set of common algorithm outputs Q_{h-1} is smaller than w_d, then the set P_h containing the most common elements of T_y for $y \in \text{SEQS}(Q_{h-1}, t_d)$ can not be entirely contained by Q_{h-1}: if we did have $P_h \subseteq Q_{h-1}$, then one can show it is possible to construct an impossibly efficient protocol for AVOID$(|Q_{h-1}|, t_d, w_{d-1})$. Consequently, until for some value of h we have $|Q_h| \geq w_d$, it is possible to find a slightly larger Q_{h+1}. More detail is given in the proof of Theorem 2.4.

Before beginning the formal proof, let us set the various parameters precisely.

Concretely, we set

$$p = \left\lceil \max\left(\sqrt{\frac{10\ell \log(64|S|)}{3z \log \frac{1}{2\delta}}}, \frac{30 \log(64|S|)}{\log \frac{1}{2\delta}} \right) \right\rceil . \tag{3}$$

$$d = 1 + \left\lfloor \frac{\ell}{18zp} \right\rfloor , \tag{4}$$

$$t_d = t_{d-1} = \cdots = t_2 = \lceil 4 \ln 2(zp+2) \rceil . \tag{5}$$

Recall also that

$$\varepsilon \leq (2\delta)^{p/30} . \tag{6}$$

We further define

$$\varepsilon_k := w_k (64|S|)^{k-1} \varepsilon , \tag{7}$$

which will upper-bound the probability that $\textsc{fco}(A, C, x, k) \neq \textsc{fco}(\Pi, C, x, k)$. We record some useful estimations in the next lemma.

Lemma 2.1. *With the above choices, $t_d = \cdots = t_2 \leq 9zp$, $t_1 \geq \ell/2$, and $\varepsilon \leq 1/(64|S|)^d$.*

Proof. The claim about the t_is holds since $z \geq 1$ and $p \geq 1$. Using this in eq. (4) gives $t_1 \geq \ell - (d-1)9zp \geq \ell/2$. We turn to proving the claim about ε. Recall that, by design, $\varepsilon \leq (2\delta)^{p/30}$.

The two branches of the maximum in eq. (3) ensure that:

$$p^2 \geq \frac{10\ell \log(64|S|)}{3z \log \frac{1}{2\delta}} \qquad \text{and} \qquad p \geq \frac{30 \log(64|S|)}{\log \frac{1}{2\delta}} . \tag{8}$$

Because $d \leq \max(1, \frac{\ell}{9zp})$, it suffices to prove that $\varepsilon \leq 1/(64|S|)$ and that $\varepsilon \leq 1/(64|S|)^{\ell/(9zp)}$. Now,

$$\log \frac{1}{\varepsilon} \underset{\text{Theorem 1.2}}{\geq} \frac{p}{30} \log \frac{1}{2\delta} \underset{\text{eq. (8)}}{\geq} \frac{1}{30p} \frac{10\ell \log(64|S|)}{3z} = \frac{\ell}{9zp} \log(64|S|) ;$$

$$\log \frac{1}{\varepsilon} \underset{\text{Theorem 1.2}}{\geq} \frac{p}{30} \log \frac{1}{2\delta} \underset{\text{eq. (8)}}{\geq} \log(64|S|) . \qquad \square$$

2.1 Common Outputs Behave Canonically

We now come to the central lemma in the proof, which asserts that the set of common outputs is likely the same for the canonical function Π as it is for a random draw $A \sim \mathcal{B}$. It also asserts two other key properties of FCO. The lemma can be thought of as a "proof of correctness" of FCO.

Lemma 2.2. *Let $k \in [d]$ and $x \in [n]^{t_d + \cdots + t_{k+1}}$. Then FCO satisfies the following properties.*

1. $\Pr_{A \sim \mathcal{B}, C \in_R [1,2)^{d \times \mathbb{N}}} [\textsc{fco}(A, C, x, k) = \textsc{fco}(\Pi, C, x, k)] \geq 1 - \varepsilon_k$.

2. *For all $C \in [1,2)^d$, the set $\textsc{fco}(\Pi, C, x, k)$ is disjoint from x and a subset of S.*

3. *For all $A\colon [n]^\ell \to [n]$ and $C \in [1,2)^d$, $\textsc{fco}(A, C, x, k)$ outputs a set of size w_k.*

Proof. The proof is by induction on k, spread over the next few lemmas. The case $k = 1$ is handled in Theorem 2.3. For the induction step, the heart of the argument, which invokes a communication lower bound for AVOID, is given in Theorem 2.4. Following this, Theorem 2.5 establishes the latter two claims in the lemma and Theorem 2.6 establishes the first claim. □

Lemma 2.3. *Theorem 2.2 holds for $k = 1$.*

Proof. Let $e_0, \ldots, e_{t_1}$ be the values of the variables on Lines 4 to 6 of Listing 1 when $\textsc{fco}(\Pi, C, x, 1)$ is called; note that these do not depend on C. For $i \in \{0, \ldots, t_1\}$, define sequences $s_i = \langle e_0, \ldots, e_{i-1}, 1, \ldots, 1\rangle$, so that $s_0 = \langle 1, 1, \ldots, 1\rangle$, and $s_{t_1} = \langle e_0, \ldots, e_{t_1-1}\rangle$. If, for all $i \in \{0, \ldots, t_1\}$, we have $A(x \circ s_i) = \Pi(x \circ s_i)$, the value of $\textsc{fco}(A, C, x, 1)$ will exactly match $\textsc{fco}(\Pi, C, x, 1)$. By a union bound,

$$\Pr_{A\sim\mathcal{B},C}[\textsc{fco}(A, C, x, k) \neq \textsc{fco}(\Pi, C, x, k)] \leq \sum_{i=0}^{t_1} \Pr_{A\sim\mathcal{B}}[A(x \circ s_i) \neq \Pi(x \circ s_i)] \underset{\text{eq. (1)}}{\leq} (t_1 + 1)\varepsilon = \varepsilon_1 .$$

Because Π is the canonical output function for a protocol for MIF, for any $z \in [n]^\ell$, we have $\Pi(z) \notin z$. Consequently, each $e_i = \Pi(x \circ \langle e_0, \ldots, e_{i-1}, 1, \ldots, 1\rangle)$ is neither contained in x nor by $\{e_0, \ldots, e_{i-1}\}$; thus $\{e_0, \ldots, e_{t_1}\}$ has size $t_1+1 = w_1$ and is disjoint from x. □

Lemma 2.4. *Let $x \in [n]^{t_d+\cdots+t_{k+1}}$. When computing $\textsc{fco}(\Pi, C, x, k)$, in the hth loop iteration, if $|Q_{h-1}| < w_k$, then $|P_h \setminus Q_{h-1}| \geq \lceil \frac{1}{4} w_{k-1} \rceil$. Consequently, the algorithm will return using Algorithm 1, not Algorithm 1.*

Proof. Assume for sake of contradiction that $|Q_{h-1}| < w_k$ and $|P_h \setminus Q_{h-1}| \leq \lfloor \frac{1}{4} w_{k-1} \rfloor$. Then we can use the algorithm $\mathcal{A}$ to implement a protocol for the one-way communication problem $\textsc{avoid}(|Q_{h-1}|, t_k, \lceil \frac{1}{2} w_{k-1} \rceil)$, with $\leq \frac{1}{2}$ probability of error.

We assume without loss of generality that $Q_{h-1} = [|Q_{h-1}|]$; if not, relabel coordinates so that this holds. In the protocol, after Alice is given a subset $W \subseteq Q_{h-1}$ with $|W| = t_k$, they construct a sequence $v = x \circ \textsc{sort}(W)$ in $[n]^{t_d+\cdots+t_k}$. Then Alice uses public randomness to instantiate an instance E of $\mathcal{A}$; inputs the sequence v to E; and sends the new state of E to Bob, using a zp-bit message. As Bob shares the public randomness, they can use this state to evaluate the output of the algorithm on any continuation of the stream. In particular, Bob can evaluate the algorithm for any possible suffix, to produce a function $\tilde{A}_{x\circ\textsc{sort}(W)} : [n]^{t_{k-1}+\cdots+t_1} \to [n]$; Bob then samples a random $C \in [1,2)^{d\times\mathbb{N}}$, and computes $V = \textsc{fco}(\tilde{A}_{x\circ\textsc{sort}(W)}, C, k-1)$, which is a subset of S. If $|V \cap Q_{h-1}| \geq \lceil \frac{1}{2} w_{k-1} \rceil$, Bob outputs the smallest $\lceil \frac{1}{2} w_{k-1} \rceil$ entries of $V \cap Q_{h-1}$. Otherwise, Bob outputs an arbitrary set of size $\lceil \frac{1}{2} w_{k-1} \rceil$.

First, we observe that for any value of $\textsc{sort}(W)$, the distribution of $\tilde{A}_{x\circ\textsc{sort}(W)}$ is exactly the same as the distribution of $A_{x\circ\textsc{sort}(W)}$, when A is drawn from $\mathcal{B}$;

this follows because for a fixed setting of the oracle random string of the algorithm, it behaves deterministically.

Applying Theorem 2.2 at $k-1$, we observe that for any $W \in \binom{Q_{h-1}}{t_k}$,

$$\Pr[\textsc{fco}(\tilde{A}_{x\circ\textsc{sort}(W)}, C, k-1) = \textsc{fco}(\Pi, C, x \circ \textsc{sort}(W), k-1)] \geq 1 - \varepsilon_{k-1} \geq \frac{3}{4}\,.$$

Furthermore, we are guaranteed that $\textsc{fco}(\Pi, C, x \circ \textsc{sort}(W), k-1)$ has size w_{k-1} and is disjoint from W.

We now bound the probability, over a uniformly random $y \in \textsc{seqs}(Q_{h-1}, t_k)$, that $|\textsc{fco}(\Pi, C, x\circ y, k-1) \cap Q_{h-1}| < \lceil \frac{1}{2} w_{k-1} \rceil$. Define $T_y = \textsc{fco}(\Pi, C, x\circ y, k-1)$ and, for each $j \in S$, $f_j^{(h)}$, as in Listing 1. In particular, we have:

$$\begin{aligned}
\Pr_{y,C}\left[|T_y \cap Q_{h-1}| < \left\lceil \frac{1}{2} w_{k-1} \right\rceil\right] &= \Pr_{y,C}\left[|T_y \setminus Q_{h-1}| > \left\lfloor \frac{1}{2} w_{k-1} \right\rfloor\right] \\
&\leq \Pr_{y,C}\left[|T_y \setminus P_h \setminus Q_{h-1}| > \left\lfloor \frac{1}{2} w_{k-1} \right\rfloor - \left\lfloor \frac{1}{4} w_{k-1} \right\rfloor\right] \qquad (9) \\
&\leq \Pr_{y,C}\left[|T_y \setminus P_h| \geq \frac{1}{4} w_{k-1}\right]\,.
\end{aligned}$$

(The inequality on eq. (9) follows since we assumed $|P_h \setminus Q_{h-1}| \leq \lfloor \frac{1}{4} w_{k-1} \rfloor$.) Note that:

$$\sum_{j \notin P_h} f_j^{(h)} = \sum_{y \in \textsc{seqs}(Q_{h-1}, t_k)} |T_y \setminus P_h| \geq \frac{1}{4} w_{k-1} \left| \left\{ y \in \textsc{seqs}(Q_{h-1}, t_k) : |T_y \setminus P_h| \geq \frac{1}{4} w_{k-1} \right\} \right|\,. \qquad (10)$$

Using the fact that y is uniformly distributed over $\textsc{seqs}(Q_{h-1}, t_k)$, gives:

$$\begin{aligned}
\Pr_{y,C}\left[|T_y \setminus P_h| \geq \frac{1}{4} w_{k-1}\right] &= \mathbb{E}_C \frac{\left|\left\{y \in \textsc{seqs}(Q_{h-1}, t_k) : |T_y \setminus P_h| \geq \frac{1}{4} w_{k-1}\right\}\right|}{\binom{|Q_{h-1}|}{t_k}} \\
&\leq \mathbb{E}_C \frac{\sum_{j \notin P_h} f_j^{(h)}}{\frac{1}{4} w_{k-1} \binom{|Q_{h-1}|}{t_k}} && \text{by eq. (10)} \\
&\leq \mathbb{E}_C \frac{(|S| - |P_h|) \frac{C_{k,h} w_{k-1}}{16|S|} \binom{|Q_{h-1}|}{t_k}}{\frac{1}{4} w_{k-1} \binom{|Q_{h-1}|}{t_k}} && \text{by definition of } P_h \\
&= \mathbb{E}_C \frac{C_{k,h}}{4} \frac{|S| - |P_h|}{|S|} \leq \frac{1}{2}\,. && \text{since } |P_h| \geq 0,\ C_{k,h} \leq 2
\end{aligned}$$

Thus the probability that $|T_y \cap Q_{h-1}| < \lceil \frac{1}{2} w_{k-1} \rceil$ holds is $\leq 1/2$. Since Bob only gives an incorrect output when this happens or when $\textsc{fco}(\tilde{A}_{x\circ\textsc{sort}(W)}, C, k-1) \neq \textsc{fco}(\Pi, C, x \circ \textsc{sort}(W), k-1)$, it follows by a union bound that the total failure probability is $\leq \frac{1}{2} + \frac{1}{4} \leq \frac{3}{4}$.

Consequently, the protocol implementation has $\leq \frac{3}{4}$ error when inputs are drawn from the uniform distribution over $\binom{Q_{h-1}}{t_k}$; by Theorem 1.1, we obtain a lower bound on the required message length, giving

$$zp > \frac{t_k \lceil \frac{1}{2} w_{k-1} \rceil}{|Q_{h-1}| \ln 2} + \log(1 - 3/4) \geq \frac{t_k w_{k-1}}{|Q_{h-1}| \cdot 2 \ln 2} - 2\,.$$

Rearranging this slightly and using integrality of $|Q_{h-1}|$ gives:

$$|Q_{h-1}| \geq \left\lceil \frac{t_k w_{k-1}}{2\ln 2(zp+2)} \right\rceil = \left\lceil \frac{\lceil 4\ln 2(zp+2)\rceil}{2\ln 2(zp+2)} w_{k-1} \right\rceil \geq 2w_{k-1} = w_k\,,$$

but as $|Q_{h-1}| < w_k$, this implies $w_k < w_k$, which is a contradiction; this proves that the assumption $|P_h \setminus Q_{h-1}| \leq \frac{1}{4}w_{k-1}$ must have been invalid.

Finally, we observe that since, in each iteration of the loop on Lines 14 to 22, $|Q_h| = |Q_{h-1} \cup P_h| = |Q_{h-1}| + |P_h \setminus Q_{h-1}| \geq |Q_{h-1}| + \lceil \frac{1}{4}w_{k-1} \rceil$, and we initially have $|Q_0| = w_{k-1}$, the size of Q_h (assuming we haven't returned yet) must be $\geq w_{k-1}(1 + h/4)$. By the last loop iteration (with $h = 4$), we will have $|Q_h| \geq 2w_{k-1} = w_k$. □

Lemma 2.5. *For $k > 1$, $x \in [n]^{t_d+\cdots+t_{k+1}}$, $\textsc{fco}(\Pi, C, x, k)$ is disjoint from x and a subset of S; and for all A, C, k, $\textsc{fco}(A, C, x, k)$ outputs a set of size w_k.*

Proof. By Theorem 2.2 at $k-1$, the sets $T_{A,x\circ y}$ chosen on Algorithm 1 are always subsets of S and disjoint from $x \circ y$, and hence disjoint from x. Per Theorem 2.4, FindCommonOutputs will return a subset of Q_h using Algorithm 1, where h is the last loop iteration number. Each element of Q_h was either in $T_{A,x\circ\langle 1,2,\ldots,t_k\rangle}$ (and hence also in S) or was in $P_{h'}$ for some $h' \leq h$. Note that $P_{h'}$ only contains integers j for which $f_j^{(h')} > 0$; i.e., which were contained in one of the sets $(T_{A,x\circ y})_{y\in \textsc{seqs}(Q_{h'-1},t_k)}$, and are thereby also in S. Consequently, the set returned is contained in S, which implies $|S| \geq w_k$.

Calls to $\textsc{fco}(A, C, x, k)$ will either output through Algorithm 1 (where the size of the set has been checked by the pseudocode) or through Algorithm 1 (where a subset of size w_k must exist, because we know $|S| \geq w_k$). □

Lemma 2.6. *For $k > 1$, and all $x \in [n]^{t_d+\cdots+t_{k+1}}$,*

$$\Pr_{A\sim\mathcal{D},C}[\textsc{fco}(A, C, x, k) \neq \textsc{fco}(\Pi, C, x, k)] \leq \varepsilon_k\,.$$

Proof. The proof of the lemma follows from the observation that, when computing $\textsc{fco}(A, C, x, k)$, even if a fraction of the recursive calls to $\textsc{fco}(A, C, x \circ y, k-1)$ produced incorrect outputs, the values for Q_0 and $(P_h)_{h\geq 1}$ will likely match those computed when $\textsc{fco}(\Pi, C, x, k)$ is called.

Henceforth, we indicate variables from the computation of $\textsc{fco}(\Pi, C, x, k)$ without a tilde, and variables from the computation of $\textsc{fco}(A, C, x, k)$ with a tilde. For example, $f_j^{(h)}$ is computed using $B = \Pi$, while $\tilde{f}_j^{(h)}$ is computed using $B = A$. We also define

$$\hat{f}_j^{(h)} = |\{y \in \textsc{seqs}(Q_{h-1}, t_k) : j \in T_{A,y}\}|$$
$$\hat{P}_h = \left\{ j \in S : \hat{f}_j^{(h)} \geq \frac{C_{k,h} w_{k-1}}{16|S|} \left| \binom{Q_{h-1}}{t_k} \right| \right\};$$

that is, $\hat{f}_j^{(h)}$ and $\hat{P}_h$ are the values that would be computed by $\textsc{fco}(A, C, x, k)$ if the set Q_{h-1} was used instead of the set $\widetilde{Q}_{h-1}$.

Say $\textsc{fco}(\Pi, C, x, k)$ returns from the loop at iteration $h^\star$. The output of $\textsc{fco}(A, C, x, k)$ will equal $\textsc{fco}(\Pi, C, x, k)$ if $Q_0 = \widetilde{Q}_0$ and for all $h \in [h^\star]$, we have $P_h = \hat{P}_h$. (If this occurs, then as $Q_0 = \widetilde{Q}_0$, $\hat{P}_1 = \widetilde{P}_1$, so $Q_1 = Q_0 \cup P_1 =$

$\widetilde{Q}_0 \cup \widetilde{P}_1 = \widetilde{Q}_1$, and as $Q_1 = \widetilde{Q}_1$, $\hat{P}_2 = \widetilde{P}_2$, and so on.) By Theorem 2.2 at $k-1$, the probability that $Q_0 \neq \widetilde{Q}_0$ is $\leq \varepsilon_{k-1}$. Consider a specific $h \in [h^\star]$; the only way in which $\hat{P}_h \neq P_h$ is if there is some $j \in S$ for which $f_j^{(h)}$ and $\hat{f}_j^{(h)}$ are on opposite sides of the threshold $\frac{C_{k,h} w_{k-1}}{16|S|} |\binom{Q_{h-1}}{t_k}|$.

Let λ_h be the random variable indicating the fraction of $y \in \textsc{seqs}(Q_{h-1}, t_k)$ for which $T_{A,x\circ y} \neq T_{\Pi,x\circ y}$. Note that the values $T_{A,x\circ y}$ are functions of the random variable A and of $C_{k',h}$ for $k' < k, h \in \mathbb{N}$; in particular $T_{A,x\circ y}$ is independent of $(C_{k,h})_{h\in\mathbb{N}}$. By Theorem 2.2 at $k-1$, $\Pr[T_{A,x\circ y} \neq T_{\Pi,x\circ y}] \leq \varepsilon_{k-1}$, which implies $\mathbb{E}\lambda_h \leq \varepsilon_{k-1}$.

Fix a particular setting of A and $(C_{k',h})_{k'<k,h\in\mathbb{N}}$. Since each set $T_{A,x\circ y}$ contributes 1 unit to each of w_{k-1} variables $\hat{f}_j^{(h)}$:

$$\sum_{j\in S} \left| f_j^{(h)} - \hat{f}_j^{(h)} \right| \leq w_{k-1} \left| \{ y \in \textsc{seqs}(Q_{h-1}, t_k) : T_{A,x\circ y} \neq T_{\Pi,x\circ y} \} \right| = w_{k-1} \lambda_h \binom{Q_{h-1}}{t_k} .$$

Let F be the set of possible values in $[1,2)$ for $C_{k,h}$ for which $P_h \neq \hat{P}_h$; this is a union of intervals corresponding to each pair $\left(f_j^{(h)}, \hat{f}_j^{(h)}\right)$, for $j \in S$. A given value c is bad for j if

$$f_j^{(h)} < \frac{c w_{k-1}}{16|S|} \binom{|Q_{h-1}|}{t_k} \leq \hat{f}_j^{(h)} ; \qquad \text{equivalently:} \qquad c \in \left(\frac{16|S| f_j^{(h)}}{w_{k-1}\binom{|Q_{h-1}|}{t_k}}, \frac{16|S| \hat{f}_j^{(h)}}{w_{k-1}\binom{|Q_{h-1}|}{t_k}} \right] ,$$

and similarly in the case where $\hat{f}_j^{(h)} < f_j^{(h)}$. The measure of F is:

$$\leq \sum_{j\in S} \frac{16|S|}{w_{k-1}\binom{|Q_{h-1}|}{t_k}} |\hat{f}_j^{(h)} - f_j^{(h)}| \leq \frac{16|S|}{w_{k-1}} w_{k-1} \lambda_h = 16|S|\lambda_h .$$

This upper bounds the probability that $C_{k,h} \in F$ and $P_h \neq \hat{P}_h$. We then have:

$$\Pr[P_h \neq \hat{P}_h] = \mathbb{E}_{A,(C_{k',h})_{k'<k}} \Pr[C_{k,h} \in F] \leq \mathbb{E}_{A,(C_{k',h})_{k'<k}} (16|S|\lambda_h) = 16|S|\varepsilon_{k-1} .$$

By a union bound, the probability that $Q_0 \neq \widetilde{Q}_0$ or $P_h \neq \hat{P}_h$ for any $h \leq h^\star$ is

$$\leq \varepsilon_{k-1} + h^\star 16|S|\varepsilon_{k-1} \leq (1 + 4 \cdot 16|S|)\varepsilon_{k-1} \leq \frac{64|S|w_k}{w_{k-1}} \varepsilon_{k-1} .$$

Thus $\Pr[\textsc{fco}(A, C, x, k) \neq \textsc{fco}(\Pi, C, x, k)] \leq \frac{64|S|w_k}{w_{k-1}} \varepsilon_{k-1} = \varepsilon_k$. □

We have thus completed the proof of Theorem 2.2, establishing the key properties of FCO. It is time to use them to derive our lower bound.

2.2 Obtaining a Pseudo-Deterministic Lower Bound

Lemma 2.7. *We have:*

$$z \geq \frac{\ell}{\log \frac{2|S|}{\ell}} \min \left(\frac{1}{36}, \frac{\log \frac{1}{2\delta}}{17280 \log(64|S|) \log \frac{2|S|}{\ell}} \right) .$$

Proof. A consequence of Theorem 2.2 is that $\textsc{fco}(\Pi, C, d)$ will output a set of size w_d which is a subset of S. This shows that $|S| \geq w_d$. Now, from the definition of w_d, it follows that

$$|S| \geq w_d = 2^{d-1}(t_1+1) > 2^{d-1}t_1 \geq 2^{d-1}\frac{\ell}{2} \qquad \Longrightarrow \qquad \log\frac{2|S|}{\ell} \geq d-1 = \left\lfloor \frac{\ell}{18zp} \right\rfloor.$$

Since $|S| \geq \ell+1$, the left hand side $\log\frac{2|S|}{\ell} > 1$, so using the inequality $x/2 \leq \max(\lfloor x \rfloor, 1)$ gives:

$$\frac{\ell}{36zp} \leq \log\frac{2|S|}{\ell} \qquad \Longrightarrow \qquad z \geq \frac{\ell}{36\log\frac{2|S|}{\ell}} \cdot \frac{1}{p}.$$

Next, we expand the definition of p (see eq. (3)), eliminating the ceiling using the inequality $\lceil x \rceil \leq \max(1, 2x)$:

$$z \geq \frac{\ell}{36\log\frac{2|S|}{\ell}} \min\left(1, \frac{1}{2}\sqrt{\frac{3z\log\frac{1}{2\delta}}{10\ell\log(64|S|)}}, \frac{1}{2}\cdot\frac{\log\frac{1}{2\delta}}{30\log(64|S|)}\right).$$

We have two cases: if the left or right side of the minimum is smallest, then:

$$z \geq \min\left(\frac{\ell}{36\log\frac{2|S|}{\ell}}, \frac{\ell\log\frac{1}{2\delta}}{2160\log\frac{2|S|}{\ell}\log(64|S|)}\right), \tag{11}$$

while otherwise, if the center is smallest, we get:

$$z^2 \geq \frac{1}{4}\frac{\ell^2}{(36\log\frac{2|S|}{\ell})^2}\cdot\frac{3z\log\frac{1}{2\delta}}{10\ell\log(64|S|)} \qquad \Longrightarrow \qquad z \geq \frac{\ell\log\frac{1}{2\delta}}{17280(\log\frac{2|S|}{\ell})^2\log(64|S|)}.$$

As $\log\frac{2|S|}{\ell} \geq 1$, this is smaller than the right minimum branch of eq. (11), so the common lower bound for all cases is:

$$z \geq \min\left(\frac{\ell}{36\log\frac{2|S|}{\ell}}, \frac{\ell\log\frac{1}{2\delta}}{17280(\log\frac{2|S|}{\ell})^2\log(64|S|)}\right). \qquad \square$$

Lemma 2.8. *We have* $|S| < 2^{z+1}$.

Proof. For each $a \in S$, let $x_a \in \Pi^{-1}(a)$. One can use $\mathcal{A}$ to provide a randomized $\leq \delta$-error, z-bit encoding of the elements in S. Using public randomness, encoder and decoder choose the oracle random string for $\mathcal{A}$. Each $a \in S$ is encoded by sending x_a to $\mathcal{A}$ and outputting the algorithm state σ. To decode, given a state σ, one evaluates the output of $\mathcal{A}$ at state σ. Using the minimax principle, one can prove that the randomized encoding requires $\geq \log((1-\delta)|S|)$ bits of space, which implies $2^z \geq (1-\delta)|S|$. Since $\delta \leq \frac{1}{3}$, it follows $s \leq \frac{3}{2}2^z < 2^{z+1}$. $\square$

We now establish the main result.

Theorem 2.9. *Pseudo-deterministic δ-error random oracle algorithms for $\textsc{mif}(n, \ell)$ require*

$$\Omega\left(\min\left(\frac{\ell}{\log\frac{2n}{\ell}} + \sqrt{\ell}, \frac{\ell\log\frac{1}{2\delta}}{(\log\frac{2n}{\ell})^2\log n} + \left(\ell\log\frac{1}{2\delta}\right)^{1/4}\right)\right)$$

bits of space when $\delta \leq \frac{1}{3}$. In particular, when $\delta = 1/\text{poly}(n)$ and $\ell = \Omega(\log n)$, this is:

$$\Omega\left(\frac{\ell}{(\log \frac{2n}{\ell})^2} + (\ell \log n)^{1/4}\right).$$

Proof sketch. Using Theorem 2.8 and the fact that $S \subseteq [n]$, we obtain $|S| \leq \min(n, 2^{z+1})$. The theorem follows by combining this bound with the inequality of Theorem 2.7, and for each of four cases corresponding to different branches of min and max, solving to find a lower bound on z. The full proof with calculations is given in Section 2.3. □

2.3 Mechanical Proofs for Section 2

Proof of Theorem 2.9. By Theorem 2.7, we have:

$$z \geq \min\left(\frac{\ell}{36 \log \frac{2|S|}{\ell}}, \frac{\ell}{17280 \log \frac{2|S|}{\ell}} \cdot \frac{\log(1/2\delta)}{\log(64|S|) \log \frac{2|S|}{\ell}}\right). \tag{12}$$

By Theorem 2.8, $|S| \leq \min(n, 2^{z+1}) \leq \min(n, 4^z)$. We will apply this inequality to each branch of the minimum in Eq. 12. First, say that the left part of the minimum is larger than the right. Then $z \geq \ell/(36 \log \frac{2|S|}{\ell})$. Applying $|S| \leq n$ and $|S| \leq 4^z$, this implies:

$$z \geq \frac{\ell}{36 \log \frac{2n}{\ell}}, \qquad \text{and}$$

$$z \geq \frac{\ell}{36 \log \frac{2 \cdot 4^z}{\ell}} \geq \frac{\ell}{36 \cdot 3z} \qquad \Longrightarrow \qquad z \geq \sqrt{\frac{\ell}{108}}.$$

Thus:

$$z \geq \max\left(\frac{\ell}{36 \log \frac{2n}{\ell}}, \sqrt{\frac{\ell}{108}}\right). \tag{13}$$

Next, say that the right side of the minimum in Eq. 12 is larger. Then applying $|S| \leq n$ and $|S| \leq 4^z$ to that side, we get:

$$z \geq \frac{\ell \log(1/2\delta)}{17280(\log \frac{2n}{\ell})^2 \log(64n)}, \qquad \text{and}$$

$$z \geq \frac{\ell \log(1/2\delta)}{17280(\log \frac{2 \cdot 4^z}{\ell})^2 \log(64 \cdot 4^z)} \geq \frac{\ell \log(1/2\delta)}{17280 \cdot 3^2 \cdot 8z^3} \qquad \Longrightarrow \qquad z \geq \left(\frac{\ell \log(1/2\delta)}{1244160}\right)^{1/4}.$$

Thus:

$$z \geq \max\left(\frac{\ell \log(1/2\delta)}{17280(\log \frac{2n}{\ell})^2 \log(64n)}, \left(\frac{\ell \log(1/2\delta)}{1244160}\right)^{1/4}\right). \tag{14}$$

The minimum of the lower bounds from Eqs. 13 and 14 holds in all cases, so:

$$z \geq \min\left(\max\left(\frac{\ell}{36 \log \frac{2n}{\ell}}, \sqrt{\frac{\ell}{108}}\right), \max\left(\frac{\ell \log(1/2\delta)}{17280(\log \frac{2n}{\ell})^2 \log(64n)}, \left(\frac{\ell \log(1/2\delta)}{1244160}\right)^{1/4}\right)\right).$$

□

References

[ABJ+22] Miklós Ajtai, Vladimir Braverman, T.S. Jayram, Sandeep Silwal, Alec Sun, David P. Woodruff, and Samson Zhou. The white-box adversarial data stream model. In *Proc. 41st ACM Symposium on Principles of Database Systems*, pages 15–27, 2022.

[ACGS23] Sepehr Assadi, Amit Chakrabarti, Prantar Ghosh, and Manuel Stoeckl. Coloring in graph streams via deterministic and adversarially robust algorithms. In *Proc. 42nd ACM Symposium on Principles of Database Systems*, pages 141–153, 2023.

[BEEO22] Omri Ben-Eliezer, Talya Eden, and Krzysztof Onak. Adversarially robust streaming via dense-sparse trade-offs. In *Symposium on Simplicity in Algorithms (SOSA)*, pages 214–227, 2022.

[BJWY20] Omri Ben-Eliezer, Rajesh Jayaram, David P. Woodruff, and Eylon Yogev. A framework for adversarially robust streaming algorithms. In *Proc. 39th ACM Symposium on Principles of Database Systems*, page 63–80, 2020.

[BKKS23] Vladimir Braverman, Robert Krauthgamer, Aditya Krishnan, and Shay Sapir. Lower bounds for pseudo-deterministic counting in a stream. *arXiv preprint arXiv:2303.16287*, 2023.

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proc. 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

[BY20] Omri Ben-Eliezer and Eylon Yogev. The adversarial robustness of sampling. In *Proc. 39th ACM Symposium on Principles of Database Systems*, pages 49–62. ACM, 2020.

[CGS22] Amit Chakrabarti, Prantar Ghosh, and Manuel Stoeckl. Adversarially robust coloring for graph streams. In *Proc. 13th Conference on Innovations in Theoretical Computer Science*, pages 37:1–37:23, 2022.

[Fei19] Uriel Feige. A randomized strategy in the mirror game. *arXiv preprint arXiv:1901.07809*, 2019.

[GGMW20] Shafi Goldwasser, Ofer Grossman, Sidhanth Mohanty, and David P. Woodruff. Pseudo-Deterministic Streaming. In *Proc. 20th Conference on Innovations in Theoretical Computer Science*, volume 151, pages 79:1–79:25, 2020.

[GGS23] Ofer Grossman, Meghal Gupta, and Mark Sellke. Tight space lower bound for pseudo-deterministic approximate counting. *arXiv preprint arXiv:2304.01438*, 2023.

[GS18] Sumegha Garg and Jon Schneider. The Space Complexity of Mirror Games. In *Proc. 10th Conference on Innovations in Theoretical Computer Science*, pages 36:1–36:14, 2018.

[HKM+20] Avinatan Hassidim, Haim Kaplan, Yishay Mansour, Yossi Matias, and Uri Stemmer. Adversarially robust streaming algorithms via differential privacy. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[HW13] Moritz Hardt and David P. Woodruff. How robust are linear sketches to adaptive inputs? In *Proc. 45th Annual ACM Symposium on the Theory of Computing*, pages 121–130, 2013.

[Ind06] Piotr Indyk. Stable distributions, pseudorandom generators, embeddings, and data stream computation. *J. ACM*, 53(3):307–323, 2006.

[JP83] Kumar Joag-Dev and Frank Proschan. Negative association of random variables, with applications. *Ann. Stat.*, 11(1):286–295, 1983.

[JW23] Rajesh Jayaram and David P Woodruff. Towards optimal moment estimation in streaming and distributed models. *ACM Trans. Alg.*, 19(3):1–35, 2023.

[KMNS21] Haim Kaplan, Yishay Mansour, Kobbi Nissim, and Uri Stemmer. Separating adaptive streaming from oblivious streaming using the bounded storage model. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 94–121. Springer, 2021.

[MN22a] Roey Magen and Moni Naor. Mirror games against an open book player. In *11th International Conference on Fun with Algorithms (FUN 2022)*, volume 226, pages 20:1–20:12, 2022.

[MN22b] Boaz Menuhin and Moni Naor. Keep that card in mind: Card guessing with limited memory. In *Proc. 13th Conference on Innovations in Theoretical Computer Science*, pages 107:1–107:28, 2022.

[New91] Ilan Newman. Private vs. common random bits in communication complexity. *Inform. Process. Lett.*, 39(2):67–71, 1991.

[Nis90] Noam Nisan. Pseudorandom generators for space-bounded computation. In *Proc. 22nd Annual ACM Symposium on the Theory of Computing*, pages 204–212, 1990.

[Nis93] Noam Nisan. On read once vs. multiple access to randomness in logspace. *Theoretical Computer Science*, 107(1):135–144, 1993.

[Sto23] Manuel Stoeckl. Streaming algorithms for the missing item finding problem. In *Proc. 34th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 793–818, 2023. Full version at arXiv:2211.05170v1.

[WZ22] David P. Woodruff and Samson Zhou. Tight bounds for adversarially robust streams and sliding windows via difference estimators. In *Proc. 62nd Annual IEEE Symposium on Foundations of Computer Science*, pages 1183–1196, 2022.